

Язык программирования C++

Лекция 14: шаблоны, исключения, раскрутка стека, гарантии при работе с исключениями

Никита Лисица

2026

Шаблонные классы

```
// array.h

template <typename T>
class Array {
    T* data;
    size_t size;

public:
    Array(size_t size)
        : data(new T[size])
        , size(size)
    {}

    ~Array() {
        delete[] data;
    }

    T& operator[] (size_t index) {
        return data[index];
    }
};
```

- Реализации методов можно вынести наружу:

```
// array.h

template <typename T>
class Array {
    T* data;
    size_t size;

public:
    Array(size_t size);
    ~Array();
    T& operator[] (size_t index);
};

...
```

Шаблонные классы

- Реализации методов можно вынести наружу:

```
// array.h

...

template <typename T>
Array<T>::Array(size_t size)
    : data(new T[size])
    , size(size)
{}

template <typename T>
Array<T>::~~Array() {
    delete[] data;
}

template <typename T>
T& Array<T>::operator[] (size_t index) {
    return data[index];
}
```

```
#include <array.h>

int main() {
    Array<int> array1;
    Array<double> array2;
    Array<Array<int>> array3;
}
```

```
#include <array.h>

int main() {
    Array<int> array1;
    Array<double> array2;
    Array<Array<int>> array3;
}
```

- Подстановку делает **компилятор**, а не препроцессор!

```
#include <array.h>

int main() {
    Array<int> array1;
    Array<double> array2;
    Array<Array<int>> array3;
}
```

- Подстановку делает **компилятор**, а не **препроцессор**!
- Код шаблонного класса должен быть в заголовочном файле. Почему?

```
#include <array.h>

int main() {
    Array<int> array1;
    Array<double> array2;
    Array<Array<int>> array3;
}
```

- Подстановку делает **компилятор**, а не **препроцессор**!
- Код шаблонного класса должен быть в заголовочном файле. Почему? Мы не знаем заранее, какие именно типы могут подставить в качестве T


```
#include <array.h>

int main() {
    Array<int> array1;
    Array<double> array2;
    Array<Array<int>> array3;
}
```

- Подстановку делает **компилятор**, а не **препроцессор**!
- Код шаблонного класса должен быть в заголовочном файле. Почему? Мы не знаем заранее, какие именно типы могут подставить в качестве T
- Методы шаблонного класса всегда **inline**. Почему?

```
#include <array.h>

int main() {
    Array<int> array1;
    Array<double> array2;
    Array<Array<int>> array3;
}
```

- Подстановку делает **компилятор**, а не **препроцессор**!
- Код шаблонного класса должен быть в заголовочном файле. Почему? Мы не знаем заранее, какие именно типы могут подставить в качестве **T**
- Методы шаблонного класса всегда **inline**. Почему? Один и тот же тип **T** могут подставить в разных **cpp**-файлах

- T называется *шаблонным параметром*

- T называется *шаблонным параметром*
- `template <typename T>` и `template <class T>` ничем не отличаются (лучше `typename` – шаблонный параметр не обязан быть классом)

- T называется *шаблонным параметром*
- `template <typename T>` и `template <class T>` ничем не отличаются (лучше `typename` – шаблонный параметр не обязан быть классом)
- `template <struct T>` нельзя :)

- Т называется *шаблонным параметром*
- `template <typename T>` и `template <class T>` ничем не отличаются (лучше `typename` – шаблонный параметр не обязан быть классом)
- `template <struct T>` нельзя :)
- Класс с конкретными подставленными значениями (например, `Array<int>`) называется *специализацией* (*specialization*) шаблона

Шаблонные классы

- Т называется *шаблонным параметром*
- `template <typename T>` и `template <class T>` ничем не отличаются (лучше `typename` – шаблонный параметр не обязан быть классом)
- `template <struct T>` нельзя :)
- Класс с конкретными подставленными значениями (например, `Array<int>`) называется *специализацией* (*specialization*) шаблона
- Можно делать синонимы типов на специализации:
`using IntArray2D = Array<Array<int>>;`

Шаблонные классы

- Т называется *шаблонным параметром*
- `template <typename T>` и `template <class T>` ничем не отличаются (лучше `typename` – шаблонный параметр не обязан быть классом)
- `template <struct T>` нельзя :)
- Класс с конкретными подставленными значениями (например, `Array<int>`) называется *специализацией* (*specialization*) шаблона
- Можно делать синонимы типов на специализации:
`using IntArray2D = Array<Array<int>>;`
- Процесс генерации кода для конкретной специализации называется *инстанцированием* (*instantiation*) шаблона

Несколько шаблонных параметров

```
template <typename Key, typename Value>
class RedBlackTree {
    struct Node {
        Key key;
        Value value;
    };

    Node* root;

public:
    ...
};
```

Значения по умолчанию

```
template <typename T = float>
struct Vector3 {
    T x, y, z;
};

Vector3 v1; // Vector3<float>
Vector3<int> v2;
Vector3<double> v3;
```

Шаблонные функции

```
template <typename T>
void swap(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}

int main() {
    int i = 10;
    int j = 20;
    swap<int>(i, j);
}
```

Шаблонные функции

```
template <typename T>
void swap(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}

int main() {
    int i = 10;
    int j = 20;
    swap<int>(i, j);
}
```

- Шаблонные функции тоже должны быть в заголовочных файлах!

Шаблонные функции

- Типы аргументов могут зависеть от шаблонных параметров:

```
template <typename T>
void reverse(Array<T>& array)
{
    size_t size = array.size();
    for (size_t i = 0; i < size / 2; ++i) {
        swap<T>(array[i], array[size - i - 1]);
    }
}

int main() {
    Array<int> array(10);
    ...
    reverse<int>(array);
}
```

Несколько шаблонных параметров

```
template <typename T, typename H>
void copy(const Array<T>& src, Array<H>& dst)
    dst.resize(src.size());
    for (size_t i = 0; i < src.size(); ++i)
        dst[i] = src[i];
}

int main() {
    Array<int> array(10);
    ...
    Array<double> array2;
    copy<int, double>(array, array2);
}
```

Вывод шаблонных параметров

- Часто компилятор может сам вывести шаблонные параметры из типов аргументов:

```
Array<int> array(10);  
// Array<T> = Array<int>  
// => T = int  
reverse(array);  
  
Array<double> array2;  
// Array<T> = Array<double>  
// => T = double  
reverse(double);  
  
// Array<T> = Array<int>  
// Array<H> = Array<double>  
// => T = int  
// => H = double  
copy(array, array2);
```

```
void sort(void* array, size_t size, size_t count, cmp_fn cmp);
```

- **Работает** для примитивных типов, **нет** проверок типов, **нельзя** заинлайнить сравнение

C vs ООП vs Шаблоны

```
void sort(void* array, size_t size, size_t count, cmp_fn cmp);
```

- **Работает** для примитивных типов, **нет** проверок типов, **нельзя** заинлайнить сравнение

```
void sort(Comparable** array, size_t size);
```

- **Не работает** для примитивных типов, **частичная** проверка типов, **нельзя** заинлайнить сравнение

C vs ООП vs Шаблоны

```
void sort(void* array, size_t size, size_t count, cmp_fn cmp);
```

- **Работает** для примитивных типов, **нет** проверок типов, **нельзя** заинлайнить сравнение

```
void sort(Comparable** array, size_t size);
```

- **Не работает** для примитивных типов, **частичная** проверка типов, **нельзя** заинлайнить сравнение

```
template <typename T>  
void sort(T* array, size_t size);
```

- **Работает** для примитивных типов, **есть** проверки типов, **можно** заинлайнить сравнение

```
template <typename T>  
void sort(T* array, size_t size);
```

```
template <typename T>  
void sort(T* array, size_t size);
```

- Для каждого типа `T` компилятор сгенерирует *отдельную функцию*

```
template <typename T>  
void sort(T* array, size_t size);
```

- Для каждого типа `T` компилятор сгенерирует *отдельную функцию*
- $\implies$ В неё можно заинлайнить (подставить) код сравнения

```
template <typename T>  
void sort(T* array, size_t size);
```

- Для каждого типа `T` компилятор сгенерирует *отдельную функцию*
- $\implies$ В неё можно заинлайнить (подставить) код сравнения
- $\implies$ Её можно оптимизировать отдельно, зная особенности типа `T`

```
template <typename T>  
void sort(T* array, size_t size);
```

- Для каждого типа `T` компилятор сгенерирует *отдельную функцию*
- $\implies$ В неё можно заинлайнить (подставить) код сравнения
- $\implies$ Её можно оптимизировать отдельно, зная особенности типа `T`
- $\implies$ Больше работы компилятору :)

Non-type template parameters

- Аргументы шаблонов могут быть не только типами, но и значениями (примитивных типов; с C++20 – простых структур)

```
template <size_t S>
class Bitset {
    uint8_t data[(S + 7) / 8];

public:
    bool get(size_t index) const {
        // Что-то с index/8 и index%8
    }

    void set(size_t index, bool value) {
        // Что-то с index/8 и index%8
    }
};
```


Non-type template parameters

- Аргументы шаблонов могут быть не только типами, но и значениями (примитивных типов; с C++20 – простых структур)

```
template <typename T, size_t N>
struct Vector {
    T coords[N];

    Vector operator* (T x) const {
        ...
    }

    Vector operator+ (const Vector<T, N>& v) const {
        ...
    }
};
```

Template template parameters

- Хотим реализовать стек, но позволить пользователю выбрать структуру данных для хранения элементов:

```
template <typename T, typename Container>
struct Stack {
    Container data;

public:
    void push(const T& value);
    bool empty() const;
    T pop();
};

Stack<int, Array<int>> stack1;
Stack<double, List<double>> stack2;
```

Template template parameters

- Хотим реализовать стек, но позволить пользователю выбрать структуру данных для хранения элементов:

```
template <typename T, typename Container>
struct Stack {
    Container data;

public:
    void push(const T& value);
    bool empty() const;
    T pop();
};

Stack<int, Array<int>> stack1;
Stack<double, List<double>> stack2;
```

- В чём проблема?

Template template parameters

- Хотим реализовать стек, но позволить пользователю выбрать структуру данных для хранения элементов:

```
template <typename T, typename Container>
struct Stack {
    Container data;

public:
    void push(const T& value);
    bool empty() const;
    T pop();
};

Stack<int, Array<int>> stack1;
Stack<double, List<double>> stack2;
```

- В чём проблема? Повторение кода + небезопасно:

```
// Потеря точности!
Stack<double, Array<int>> stack;
```

Template template parameters

- Решение – шаблонные шаблонные параметры:

```
template <typename T, template <typename> class Container>
struct Stack {
    Container<T> data;

public:
    ...
};

Stack<int, Array> stack1;
Stack<double, List> stack2;
```

Template template parameters

- Можно сделать значение параметра по умолчанию:

```
template <typename T,  
    template <typename> class Container = Deque  
>  
struct Stack { ... };  
  
Stack<char*> stack; // Stack<char*, Deque>
```

Специализация шаблонов (полная)

- Хотим сделать специфичное поведение для конкретного набора шаблонных параметров:

```
template <typename T>
class Array { ... };

template <>
class Array<bool> {
    // Оптимизация: храним bool'ы в битах,
    // а не отдельными объектами
    uint8_t* data;
    size_t size;

public:
    Array(size_t size)
        : data(new uint8_t[(size + 7) / 8])
        , size(size)
    {}

    ...
};
```

Специализация шаблонов (полная)

- Хотим сделать специфичное поведение для конкретного набора шаблонных параметров:

```
template <typename T, size_t N>
struct Vector { ... };

template <>
struct Vector<float, 2> {
    // Оптимизация: используем SIMD intrinsics
    float coords __attribute__((vector_size(8)));
};

template <>
struct Vector<float, 4> {
    // Оптимизация: используем SIMD intrinsics
    float coords __attribute__((vector_size(16)));
};
```


Специализация шаблонов (частичная)

- Частичная специализация – для некоторого множества параметров:

```
template <typename T>
class Array<Array<T>> { ... };

template <size_t N>
struct Vector<float, N> { ... };
```

- Ошибки – неотъемлемая часть программирования

- Ошибки – неотъемлемая часть программирования
- Ошибки по вине программиста (передали не тот параметр, вызвали не тот метод, etc):

- Ошибки – неотъемлемая часть программирования
- Ошибки по вине программиста (передали не тот параметр, вызвали не тот метод, etc):
 - Выявляют на стадии разработки / тестирования

- Ошибки – неотъемлемая часть программирования
- Ошибки по вине программиста (передали не тот параметр, вызвали не тот метод, etc):
 - Выявляют на стадии разработки / тестирования
 - В 'идеальной' программе их не должно быть (но всегда есть)

- Ошибки – неотъемлемая часть программирования
- Ошибки по вине программиста (передали не тот параметр, вызвали не тот метод, etc):
 - Выявляют на стадии разработки / тестирования
 - В 'идеальной' программе их не должно быть (но всегда есть)
 - В C и C++ их часто не обрабатывают

- Ошибки – неотъемлемая часть программирования
- Ошибки по вине программиста (передали не тот параметр, вызвали не тот метод, etc):
 - Выявляют на стадии разработки / тестирования
 - В 'идеальной' программе их не должно быть (но всегда есть)
 - В С и С++ их часто не обрабатывают
- Ошибки по вине окружения / пользователя / входных данных (файл не существует, неверный формат ввода, etc)

- Ошибки – неотъемлемая часть программирования
- Ошибки по вине программиста (передали не тот параметр, вызвали не тот метод, etc):
 - Выявляют на стадии разработки / тестирования
 - В 'идеальной' программе их не должно быть (но всегда есть)
 - В С и С++ их часто не обрабатывают
- Ошибки по вине окружения / пользователя / входных данных (файл не существует, неверный формат ввода, etc)
 - Могут произойти в любой программе

- Ошибки – неотъемлемая часть программирования
- Ошибки по вине программиста (передали не тот параметр, вызвали не тот метод, etc):
 - Выявляют на стадии разработки / тестирования
 - В 'идеальной' программе их не должно быть (но всегда есть)
 - В С и С++ их часто не обрабатывают
- Ошибки по вине окружения / пользователя / входных данных (файл не существует, неверный формат ввода, etc)
 - Могут произойти в любой программе
 - Их **обязательно** нужно обрабатывать!

Действия по обработке ошибки

- Проверить наличие ошибки

```
if (!file) { ... }
```

Действия по обработке ошибки

- Проверить наличие ошибки

```
if (!file) { ... }
```

- Освободить ресурсы

```
delete[] buffer;
```

Действия по обработке ошибки

- Проверить наличие ошибки

```
if (!file) { ... }
```

- Освободить ресурсы

```
delete[] buffer;
```

- Сообщить пользователю / вызывающей функции

```
std::cerr << "Failed to open file\n";
```

Действия по обработке ошибки

- Проверить наличие ошибки

```
if (!file) { ... }
```

- Освободить ресурсы

```
delete[] buffer;
```

- Сообщить пользователю / вызывающей функции

```
std::cerr << "Failed to open file\n";
```

- Предпринять действия по восстановлению от ошибки

```
file = fopen("backup.data", "rb");
```

C-style обработка ошибок

- Через возвращаемое значение:

```
FILE* fopen(const char* filename, const char* mode) {  
    if (/* файл не найден */) {  
        return NULL;  
    }  
    if (/* отказано в доступе */) {  
        return NULL;  
    }  
    ...  
}  
  
int Array::get(int index) {  
    if (index < 0) return -1;  
}
```

C-style обработка ошибок

- Через возвращаемое значение:

```
FILE* fopen(const char* filename, const char* mode) {  
    if (/* файл не найден */) {  
        return NULL;  
    }  
    if (/* отказано в доступе */) {  
        return NULL;  
    }  
    ...  
}  
  
int Array::get(int index) {  
    if (index < 0) return -1;  
}
```

- Непонятно, что именно произошло и что с этим делать ('что-то пошло не так')

C-style обработка ошибок

- Через возвращаемое значение:

```
FILE* fopen(const char* filename, const char* mode) {  
    if (/* файл не найден */) {  
        return NULL;  
    }  
    if (/* отказано в доступе */) {  
        return NULL;  
    }  
    ...  
}  
  
int Array::get(int index) {  
    if (index < 0) return -1;  
}
```

- Непонятно, что именно произошло и что с этим делать ('что-то пошло не так')
- Не всегда есть разумное 'ошибочное' значение возвращаемого типа

C-style обработка ошибок

- Через возвращаемое значение + глобальный флаг:

```
FILE* fopen(const char* filename, const char* mode) {  
    if (/* файл не найден */) {  
        errno = ENOENT;  
        return NULL;  
    }  
    if (/* отказано в доступе */) {  
        errno = EPERM;  
        return NULL;  
    }  
    ...  
}
```

C-style обработка ошибок

- Через возвращаемое значение + глобальный флаг:

```
FILE* fopen(const char* filename, const char* mode) {  
    if (/* файл не найден */) {  
        errno = ENOENT;  
        return NULL;  
    }  
    if (/* отказано в доступе */) {  
        errno = EPERM;  
        return NULL;  
    }  
    ...  
}
```

- Нужно следить, чтобы другая функция не перезаписала `errno`

C-style обработка ошибок

- Через возвращаемое значение + глобальный флаг:

```
FILE* fopen(const char* filename, const char* mode) {  
    if (/* файл не найден */) {  
        errno = ENOENT;  
        return NULL;  
    }  
    if (/* отказано в доступе */) {  
        errno = EPERM;  
        return NULL;  
    }  
    ...  
}
```

- Нужно следить, чтобы другая функция не перезаписала `errno`
- Нельзя передать более подробную информацию об ошибке

C-style обработка ошибок

- Через возвращаемое значение + дополнительный аргумент:

```
FILE* fopen(const char* filename, const char* mode, int* error)
{
    if (/* файл не найден */) {
        *error = ENOENT;
        return NULL;
    }
    if (/* отказано в доступе */) {
        *error = EPERM;
        return NULL;
    }
    ...
}
```

C-style обработка ошибок

- Через возвращаемое значение + дополнительный аргумент:

```
FILE* fopen(const char* filename, const char* mode, int* error)
{
    if (/* файл не найден */) {
        *error = ENOENT;
        return NULL;
    }
    if (/* отказано в доступе */) {
        *error = EPERM;
        return NULL;
    }
    ...
}
```

- Перемешивается основная логика функции и обработка ошибок

Result / Either / std::expected

- Современный подход – использовать тип-сумму:

```
std::expected<FILE*, int> fopen(  
    const char* filename, const char* mode)  
{  
    if (/* файл не найден */) {  
        return std::unexpected(ENOENT);  
    }  
    if (/* отказано в доступе */) {  
        return std::unexpected(EPERM);  
    }  
    FILE* file = ...;  
    ...  
    return file;  
}
```

Result / Either / std::expected

- Современный подход – использовать тип-сумму:

```
std::expected<FILE*, int> fopen(  
    const char* filename, const char* mode)  
{  
    if (/* файл не найден */) {  
        return std::unexpected(ENOENT);  
    }  
    if (/* отказано в доступе */) {  
        return std::unexpected(EPERM);  
    }  
    FILE* file = ...;  
    ...  
    return file;  
}
```

- Самый аккуратный подход

Result / Either / std::expected

- Современный подход – использовать тип-сумму:

```
std::expected<FILE*, int> fopen(  
    const char* filename, const char* mode)  
{  
    if (/* файл не найден */) {  
        return std::unexpected(ENOENT);  
    }  
    if (/* отказано в доступе */) {  
        return std::unexpected(EPERM);  
    }  
    FILE* file = ...;  
    ...  
    return file;  
}
```

- Самый аккуратный подход
- Всё ещё перемешивается основная логика функции и обработка ошибок


```
class DivideByZeroException {
public:
    const char* what() {
        return "division by zero";
    }
};

double divide(int a, int b) {
    if (b == 0) {
        throw DivideByZeroException();
    }
    return double(a) / b;
}
```

```
try {  
    c = divide(a, b);  
}  
catch (DivideByZeroException e) {  
    // Решаем, что с этим делать  
}
```

```
try {  
    c = divide(a, b);  
}  
catch (DivideByZeroException e) {  
    // Решаем, что с этим делать  
}
```

- **throw** *бросает* исключение

```
try {  
    c = divide(a, b);  
}  
catch (DivideByZeroException e) {  
    // Решаем, что с этим делать  
}
```

- **throw** *бросает* исключение
- **catch** *ловит* исключение

Исключения

- Брошенное исключение идёт вверх по стеку вызовов, пока не наткнётся на подходящий **try/catch** блок (*раскрутка стека – stack unwinding*):

```
int f(int x) {  
    return divide(1000, x);  
}  
  
int g(int x) {  
    return f(x - 42);  
}  
  
int main() {  
    try {  
        g(42);  
    }  
    catch (const DivideByZeroException& e) {  
        ...  
    }  
}
```

Исключения

- После бросания исключения нормальный поток исполнения останавливается – `printf` не будет вызван:

```
void f() {  
    throw Exception(...);  
    printf("I am Groot");  
}
```

Исключения

- После бросания исключения нормальный поток исполнения останавливается – `printf` не будет вызван:

```
void f() {  
    throw Exception(...);  
    printf("I am Groot");  
}
```

- Если подходящего `try/catch` блока не нашлось, программа аварийно завершается:

```
int main() {  
    // Нет try/catch  
    throw Exception(...);  
}
```

Несколько типов исключений

- Если вызываемый код может бросать несколько типов исключений, можно обработать их все:

```
int main() {  
    try {  
        runServer();  
    }  
    catch (const FileSystemError& e) {  
        ...  
    }  
    catch (const NetworkError& e) {  
        ...  
    }  
    catch (const InternalError& e) {  
        ...  
    }  
}
```


Иерархия исключений

- **catch** (Type) поймает не только тип Type, но и любого его наследника:

```
class Exception {
public:
    virtual const char* what() { return "Error"; }
};

class DivideByZeroException : public Exception { ... };

int main() {
    try {
        divide(10, 0);
    }
    catch (const Exception& e) {
        std::cerr << e.what() << std::endl;
    }
}
```

Иерархия исключений

- Часто делают иерархии классов исключений для более точной обработки ошибок:

```
class Exception { ... };

class FilesystemError : public Exception { ... };
class FileNotFoundError : public FilesystemError { ... };
class PermissionDenied : public FilesystemError { ... };

class NetworkError : public Exception { ... };
class ConnectionLost : public NetworkError { ... };
class ConnectionTimeout : public NetworkError { ... };

class InternalError : public Exception { ... };
class AssertionFailed : public InternalError { ... };
class UnreachableCode : public InternalError { ... };
```

Иерархия исключений

- Часто делают иерархии классов исключений для более точной обработки ошибок:

```
class Exception { ... };

class FilesystemError : public Exception { ... };
class FileNotFound      : public FilesystemError { ... };
class PermissionDenied : public FilesystemError { ... };

class NetworkError      : public Exception { ... };
class ConnectionLost    : public NetworkError { ... };
class ConnectionTimeout : public NetworkError { ... };

class InternalError      : public Exception { ... };
class AssertionFailed    : public InternalError { ... };
class UnreachableCode    : public InternalError { ... };
```

- В стандартной библиотеке все исключения – наследники `std::exception`

Иерархия исключений

- Важно: в списке `catch` блоков выбирается **первый подходящий!**

```
try {  
    ...  
}  
catch (const Exception& e) {  
    ...  
}  
catch (const ConnectionLost& e) {  
    // Никогда не вызовется!  
}
```

- Правильный порядок:

```
try {  
    ...  
}  
catch (const ConnectionLost& e) {  
    // Сначала наследник,  
}  
catch (const Exception& e) {  
    // потом базовый класс исключения  
}
```

Исключения

- Можно поймать исключение неизвестного типа конструкцией `catch(...)` (но тогда с ним ничего особо не сделать):

```
try {  
    runServer();  
}  
catch (...) {  
    std::cerr << "Unknown error" << std::endl;  
}
```

Исключения

- Можно поймать исключение неизвестного типа конструкцией `catch(...)` (но тогда с ним ничего особо не сделать):

```
try {  
    runServer();  
}  
catch (...) {  
    std::cerr << "Unknown error" << std::endl;  
}
```

- **NB:** Исключение может быть любым типом, но лучше так не делать :)

```
throw NULL;  
throw 42;  
throw "Oopsie woopsie";
```

Исключения

- Можно бросить пойманное исключение дальше (*rethrow*) после его обработки конструкцией **throw**; (даже если оно неизвестного типа):

```
try {
    runServer();
}
catch (const NetworkError& e) {
    std::cerr << "Network error: " << e.what() << std::endl;
    throw;
}
catch (...) {
    std::cerr << "Unknown error" << std::endl;
    throw;
}
```


Исключения и деструкторы

- В процессе раскрутки стека автоматически вызываются деструкторы всех локальных переменных:

```
void f() {  
    Array array(100);  
    if (...) {  
        // Вызовется array.~Array()  
        throw Exception(...);  
    }  
}  
  
void g() {  
    File file(...);  
    // Если бросится исключение, вызовется file.~File()  
    f();  
}
```

Исключения и деструкторы

```
void f() {  
    int* array = new int[100];  
    if (...) {  
        // Формально, вызовется деструктор array  
        // Это примитивный тип, так что деструктор ничего не делает  
        // => Утечка памяти  
        throw Exception(...);  
    }  
    delete[] array;  
}
```

- **Resource Acquisition Is Initialization** – взятие ресурса должно происходить через инициализацию

- **Resource Acquisition Is Initialization** – взятие ресурса должно происходить через инициализацию
- По существу RAI обычно сводится к освобождению ресурса в деструкторе

- **Resource Acquisition Is Initialization** – взятие ресурса должно происходить через инициализацию
- По существу RAII обычно сводится к освобождению ресурса в деструкторе
- Не сможем забыть очистить ресурсы

Идиома RAII

- **Resource Acquisition Is Initialization** – взятие ресурса должно происходить через инициализацию
- По существу RAII обычно сводится к освобождению ресурса в деструкторе
- Не сможем забыть очистить ресурсы, теперь и в случае исключений!

```
void f() {  
    Array array(100);  
    if (...) {  
        // Вызовется array.~Array()  
        // Утечки памяти нет  
        throw Exception(...);  
    }  
}
```

Исключения в конструкторе

- До конца работы конструктора объекта, объект не считается проинициализированным, и его деструктор не вызовется:

```
// Может бросить исключение
void checkSupported(const char *effect);

class AudioEffect {
    float* buffer;

public:
    AudioEffect(const char* effect) {
        buffer = new float[1024];
        // Деструктор ~AudioEffect не вызовется
        // => Утечка памяти
        checkSupported(effect);
    }

    ~AudioEffect() {
        delete[] buffer;
    }
};
```

Исключения в конструкторе

- Решение №1:

```
AudioEffect(const char* effect) {  
    buffer = new float[1024];  
    try {  
        checkSupported(effect);  
    }  
    catch (...) {  
        delete[] buffer;  
        throw;  
    }  
}
```


Исключения в конструкторе

- Решение №2 – RAI:

```
class AudioEffect {  
    Array<float> buffer;  
  
public:  
    AudioEffect(const char* effect)  
        : buffer(1024)  
    {  
        // this->buffer уже проинициализирован  
        // Деструктор buffer.~Array() будет вызван  
        checkSupported(effect);  
    }  
};
```

Исключения в конструкторе

- Что, если конструктор одного из полей бросает исключение?

```
class AudioManager {  
    FILE* input;  
    AudioEffect effect;  
  
public:  
    // Если конструктор effect бросит исключение,  
    // деструктор AudioManager не вызовется  
    // и файл input не закроется  
    AudioManager()  
        : input(fopen("meow.wav", "rb"))  
        , effect("wrum-wrum")  
    {}  
  
    ~AudioManager() {  
        fclose(input);  
    }  
};
```

Исключения в конструкторе

- Решение №1 – особый синтаксис **try** блока:

```
AudioManager() try
: input(fopen("meow.wav", "rb"))
, effect("wrum-wrum")
{
    // тело конструктора
}
catch (...) {
    fclose(input);
    throw;
}
```

Исключения в конструкторе

- Решение №1 – особый синтаксис **try** блока:

```
AudioManager() try
    : input(fopen("meow.wav", "rb"))
    , effect("wrum-wrum")
{
    // тело конструктора
}
catch (...) {
    fclose(input);
    throw;
}
```

- NB: работает и для функций

```
int main() try {
    // тело функции
}
catch (const Exception& e) {
    std::cerr << "Error: " << e.what() << std::endl;
}
```

Исключения в конструкторе

- Решение №2 – RAII:

```
class AudioManager {  
    FileStream input;  
    AudioEffect effect;  
  
public:  
    // Если конструктор effect бросит исключение,  
    // вызовется деструктор input  
    AudioManager()  
        : input("meow.wav")  
        , effect("wrum-wrum")  
    {}  
};
```

- Совет: если вам приходится вручную обрабатывать исключения в конструкторе, скорее всего, какие-то поля надо выделить в отдельные RAII-классы :)

Исключения в деструкторе

```
class Database {  
    ...  
    ~Database() {  
        // Может бросить LoggingError  
        logger.log("Database is closed");  
    }  
};
```

Исключения в деструкторе

```
class Database {  
    ...  
    ~Database() {  
        // Может бросить LoggingError  
        logger.log("Database is closed");  
    }  
};
```

- Деструктор `~Database` может вызваться в процессе раскрутки стека из-за другого исключения

Исключения в деструкторе

```
class Database {  
    ...  
    ~Database() {  
        // Может бросить LoggingError  
        logger.log("Database is closed");  
    }  
};
```

- Деструктор `~Database` может вызваться в процессе раскрутки стека из-за другого исключения
- В других языках часто поддерживаются цепочки исключений (LoggingError caused by DatabaseError)

Исключения в деструкторе

```
class Database {  
    ...  
    ~Database() {  
        // Может бросить LoggingError  
        logger.log("Database is closed");  
    }  
};
```

- Деструктор `~Database` может вызваться в процессе раскрутки стека из-за другого исключения
- В других языках часто поддерживаются цепочки исключений (`LoggingError caused by DatabaseError`)
- В C++ исключения могут быть произвольного типа $\implies$ в общем случае к ним не добавить никакую информацию

Исключения в деструкторе

```
class Database {  
    ...  
    ~Database() {  
        // Может бросить LoggingError  
        logger.log("Database is closed");  
    }  
};
```

- Деструктор `~Database` может вызваться в процессе раскрутки стека из-за другого исключения
- В других языках часто поддерживаются цепочки исключений (`LoggingError caused by DatabaseError`)
- В C++ исключения могут быть произвольного типа $\implies$ в общем случае к ним не добавить никакую информацию
- Поэтому в C++ исключение, брошенное в деструкторе, приводит к аварийному завершению программы

Исключения в деструкторе

- Решение №1 – проигнорировать исключение:

```
class Database {  
    ...  
    ~Database() {  
        try {  
            logger.log("Database is closed");  
        }  
        catch (...) {}  
    }  
};
```

Исключения в деструкторе

- Решение №1 – проигнорировать исключение:

```
class Database {  
    ...  
    ~Database() {  
        try {  
            logger.log("Database is closed");  
        }  
        catch (...) {}  
    }  
};
```

- Плохо, потому что мы не узнаем о какой-то ошибке (но так всё равно часто делают)

Исключения в деструкторе

- Решение №2 – вынести такой код из деструктора и вызывать явно:

```
class Database {  
    ...  
    void close() {  
        logger.log("Database is closed");  
    }  
  
    ~Database() {}  
};  
  
int main() {  
    Database db;  
    ...  
    db.close();  
}
```

Исключения в деструкторе

- Решение №2 – вынести такой код из деструктора и вызывать явно:

```
class Database {  
    ...  
    void close() {  
        logger.log("Database is closed");  
    }  
  
    ~Database() {}  
};  
  
int main() {  
    Database db;  
    ...  
    db.close();  
}
```

- Плохо, потому что можно забыть вызвать `close`

- Есть ли хорошее решение?

- Есть ли хорошее решение? Нет :(

- Часто функции/методы в коде на C++ предоставляют некоторые гарантии в связи с исключениями (обычно это описано в документации)

- Часто функции/методы в коде на C++ предоставляют некоторые гарантии в связи с исключениями (обычно это описано в документации)
- **Nothrow guarantee:** функция/метод не бросает исключение

Гарантии при работе с исключениями

- Часто функции/методы в коде на C++ предоставляют некоторые гарантии в связи с исключениями (обычно это описано в документации)
- **Nothrow guarantee:** функция/метод не бросает исключение
- **Strong guarantee:** в случае исключения состояние программы откатывается до состояния перед вызовом функции/метода

Гарантии при работе с исключениями

- Часто функции/методы в коде на C++ предоставляют некоторые гарантии в связи с исключениями (обычно это описано в документации)
- **Nothrow guarantee:** функция/метод не бросает исключение
- **Strong guarantee:** в случае исключения состояние программы откатывается до состояния перед вызовом функции/метода
- **Basic guarantee:** в случае исключения программа останется в некотором допустимом состоянии

Гарантии при работе с исключениями

- Часто функции/методы в коде на C++ предоставляют некоторые гарантии в связи с исключениями (обычно это описано в документации)
- **Nothrow guarantee:** функция/метод не бросает исключение
- **Strong guarantee:** в случае исключения состояние программы откатывается до состояния перед вызовом функции/метода
- **Basic guarantee:** в случае исключения программа останется в некотором допустимом состоянии
- **No guarantee:** в случае исключения $\neg_(\text{"/})_/\neg$

No guarantee

```
void process() {  
    int* buffer = new int[1024];  
    FILE* file = fopen(...);  
    if (!file) {  
        // Утечка памяти  
        throw Exception(...);  
    }  
}
```

```
void process() {  
    int* buffer = new int[1024];  
    FILE* file = fopen(...);  
    if (!file) {  
        // Утечка памяти  
        throw Exception(...);  
    }  
}
```

- Самый плохой вариант: обычно стараются давать хоть какую-нибудь гарантию, иначе кодом невозможно пользоваться корректно

Basic guarantee

```
void update(Array<int>& array) {  
    for (size_t i = 0; i < array.size(); ++i) {  
        // Может бросить FileNotFoundException  
        FileStream file(getFilename(i));  
        array[i] = file.readNextInt();  
    }  
}
```

Basic guarantee

```
void update(Array<int>& array) {  
    for (size_t i = 0; i < array.size(); ++i) {  
        // Может бросить FileNotFoundException  
        FileStream file(getFilename(i));  
        array[i] = file.readNextInt();  
    }  
}
```

- Слабая гарантия: состояние программы останется валидным (нет утечек памяти, битых ссылок, и т.п.), но уже затронутые элементы массива останутся изменёнными

Basic guarantee

```
void update(Array<int>& array) {  
    for (size_t i = 0; i < array.size(); ++i) {  
        // Может бросить FileNotFoundException  
        FileStream file(getFilename(i));  
        array[i] = file.readNextInt();  
    }  
}
```

- Слабая гарантия: состояние программы останется валидным (нет утечек памяти, битых ссылок, и т.п.), но уже затронутые элементы массива останутся изменёнными
- Обычно легко достигается грамотным применением RAII

Strong guarantee

```
void update(Array<int>& array) {  
    Array<int> newArray(array.size());  
  
    for (size_t i = 0; i < array.size(); ++i) {  
        // Может бросить FileNotFoundException  
        FileStream file(getFilename(i));  
        newArray[i] = file.readNextInt();  
    }  
  
    // Идиома copy-and-swap  
    array.swap(newArray);  
}
```

Strong guarantee

```
void update(Array<int>& array) {  
    Array<int> newArray(array.size());  
  
    for (size_t i = 0; i < array.size(); ++i) {  
        // Может бросить FileNotFoundException  
        FileStream file(getFilename(i));  
        newArray[i] = file.readNextInt();  
    }  
  
    // Идиома copy-and-swap  
    array.swap(newArray);  
}
```

- Сильная гарантия: состояние программы не изменится

Strong guarantee

```
void update(Array<int>& array) {  
    Array<int> newArray(array.size());  
  
    for (size_t i = 0; i < array.size(); ++i) {  
        // Может бросить FileNotFoundException  
        FileStream file(getFilename(i));  
        newArray[i] = file.readNextInt();  
    }  
  
    // Идиома copy-and-swap  
    array.swap(newArray);  
}
```

- Сильная гарантия: состояние программы не изменится
- Обычно тяжело обеспечить

Strong guarantee

```
void update(Array<int>& array) {  
    Array<int> newArray(array.size());  
  
    for (size_t i = 0; i < array.size(); ++i) {  
        // Может бросить FileNotFoundException  
        FileStream file(getFilename(i));  
        newArray[i] = file.readNextInt();  
    }  
  
    // Идиома copy-and-swap  
    array.swap(newArray);  
}
```

- Сильная гарантия: состояние программы не изменится
- Обычно тяжело обеспечить
- Стандартные контейнеры стараются дать сильную гарантию везде, где это возможно

Nothrow guarantee

```
int strlen(const char* str) {  
    int result = 0;  
    for (; *str != 0; ++str, ++result);  
    return result;  
}  
  
vec2f rotate(vec2f vec, float angle) {  
    float c = cos(angle);  
    float s = sin(angle);  
    return vec2f(  
        c * vec.x - s * vec.y,  
        s * vec.x + c * vec.y,  
    );  
}
```


Nothrow guarantee

```
int strlen(const char* str) {  
    int result = 0;  
    for (; *str != 0; ++str, ++result);  
    return result;  
}  
  
vec2f rotate(vec2f vec, float angle) {  
    float c = cos(angle);  
    float s = sin(angle);  
    return vec2f(  
        c * vec.x - s * vec.y,  
        s * vec.x + c * vec.y,  
    );  
}
```

- Самая сильная гарантия: обычно означает, что код в принципе никак не может бросить исключение